EventHub Database Setup Guide

🎯 Quick Start (TL;DR)

1. Install MySQL 8.0
2. Save the SQL script as `eventhub_setup.sql`
3. Run: `mysql -u root -p < eventhub_setup.sql`
4. Done! Your database is ready.

📋 Prerequisites

- MySQL 8.0 installed on your machine
- Administrative access to MySQL
- The `eventhub_setup.sql` file (provided separately)

🔧 Step 1: Install MySQL 8.0

Windows

1. Download MySQL 8.0 from: https://dev.mysql.com/downloads/installer/
2. Run installer as administrator
3. Choose "Custom" installation
4. Select: MySQL Server 8.0, MySQL Workbench, MySQL Shell
5. Set root password (remember this!)
6. Complete installation

macOS

```
# Install via Homebrew
brew install mysql@8.0
brew services start mysql@8.0

# Secure installation
mysql_secure_installation
```

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install mysql-server-8.0 mysql-client-8.0
sudo systemctl start mysql
sudo mysql_secure_installation
```

📂 Step 2: Prepare the Database Script

Save the provided SQL code as `eventhub_setup.sql` in your project folder.

The script automatically:

- ✅ Drops existing database (safe reset)
- ✅ Creates fresh `eventhub_db` database
- ✅ Creates all 20+ tables with proper relationships
- ✅ Sets up indexes for performance
- ✅ Creates views and stored procedures
- ✅ Inserts sample data (categories, badges, admin user)

🚀 Step 3: Run the Setup Script

Option A: Command Line (Recommended)

```
# Navigate to your project folder
cd /path/to/your/project

# Run the setup script
mysql -u root -p < eventhub_setup.sql

# Enter your MySQL root password when prompted
```

Option B: MySQL Workbench

1. Open MySQL Workbench
2. Connect to your local MySQL server
3. File → Open SQL Script → Select `eventhub_setup.sql`
4. Click Execute (⚡) button
5. Wait for completion

✅ Step 4: Verify Installation

Quick Verification

```
# Connect to database
mysql -u root -p eventhub_db

# Check tables were created
SHOW TABLES;

# Should show 20+ tables including:
# - users, events, clubs, rsvps
# - event_categories, payments, badges
# - notifications, audit_log, etc.
```

Detailed Verification

```
-- Check database info
SELECT 'EventHub Database ready!' AS Status;
SELECT DATABASE() AS Current_Database;
SELECT COUNT(*) AS Total_Tables
FROM information_schema.tables
WHERE table_schema = 'eventhub_db';

-- Check sample data
SELECT * FROM event_categories;
SELECT * FROM badges;
SELECT username, user_type FROM users;
```

Expected Results:

- 8 event categories (Academic, Social, Sports, etc.)
- 7 default badges (First Timer, Frequent Attendee, etc.)
- 1 admin user

🔄 Daily Development Workflow

Starting Work

```
# Start MySQL service
brew services start mysql@8.0      # macOS
sudo systemctl start mysql         # Linux
net start mysql                    # Windows

# Connect to your database
mysql -u root -p eventhub_db
```

Stopping Work

```
# Stop MySQL service (optional - saves resources)
brew services stop mysql@8.0       # macOS
sudo systemctl stop mysql          # Linux
net stop mysql                     # Windows
```

Important: Your data persists even when MySQL is stopped!

Reset Database (if needed)

```
# Re-run the setup script to start fresh
mysql -u root -p < eventhub_setup.sql
```

👥 Team Setup Instructions

For New Team Members:

1. **Get the files:**
    - This setup guide

    - `eventhub_setup.sql` script
2. **Follow Steps 1-4 above**
3. **Test your setup:**

```
-- Connect and test
mysql -u root -p eventhub_db

-- Run a test query
SELECT
    e.title,
     c.category_name,
     cl.club_name
 FROM events e
 LEFT JOIN event_categories c ON e.category_id = c.category_id
 LEFT JOIN clubs cl ON e.club_id = cl.club_id
 LIMIT 5;
```

16. **Share your connection details:**
    - Host: `localhost`
    - Port: `3306`
    - Database: `eventhub_db`
    - Username: `root` (or create app-specific user)

🎨 Database Overview

Core Tables Structure:

```
👤 Users & Authentication
├── users (students, organizers, admins)
├── clubs (organizations)
└── club_members (roles & memberships)

🏛️ Event Management
├── events (main event data)
├── event_categories (Academic, Social, etc.)
├── event_tags & event_tag_mappings
└── event_gallery (photos)

📅 User Participation
├── rsvps (event registrations)
├── event_feedback (ratings & comments)
└── payments (for paid events)

🔔 Notifications & Features
├── notifications (system messages)
├── event_reminders
├── badges & user_badges
└── audit_log (admin actions)
```

Key Features Built-in:

- ✅ User roles (student/organizer/admin)
- ✅ Event RSVP system with check-in
- ✅ Payment processing for paid events
- ✅ Badge/achievement system

- ✅ Photo galleries
- ✅ Feedback & rating system
- ✅ Tag-based event organization
- ✅ Notification system
- ✅ Admin audit logging

🔒 Production Deployment

When moving to hosted MySQL:

1. **Export your local database:**
2. `mysqldump -u root -p eventhub_db > eventhub_backup.sql`
3. **Import to production:**
4. `mysql -h your-host.com -u username -p eventhub_db < eventhub_backup.sql`
5. **Update connection strings** in your application

🛠️ Troubleshooting

Common Issues:

"Access denied for user 'root'"

```
# Reset MySQL root password
sudo mysql
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
'your_new_password';
FLUSH PRIVILEGES;
```

"Can't connect to MySQL server"

```
# Check if MySQL is running
brew services list | grep mysql                    # macOS
sudo systemctl status mysql                        # Linux
```

"Database already exists" errors

- The script handles this automatically with `DROP DATABASE IF EXISTS`
- Safe to re-run the setup script anytime

Script fails partway through

- Check MySQL error log: `/var/log/mysql/error.log`
- Ensure you have sufficient privileges
- Try running script sections individually

Getting Help:

- Check MySQL status: `sudo systemctl status mysql`
- View error logs: `tail -f /var/log/mysql/error.log`
- Test connection: `mysql -u root -p -e "SELECT 1"`

Next Steps

After setup is complete:

1. Create application database users (not root)
2. Set up environment variables for database connection
3. Test with your application code
4. Plan backup strategy
5. Consider setting up database migrations for future changes